CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.1.1. Numpy Array Operations

10:20

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

**Input Format:**

- The first line contains two space-separated integers, $r$ and $c$, representing the number of rows and the number of columns.
- The next $r$ lines contain $c$ space-separated integers representing the elements of the array, entered row by row.

**Output Format:**

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

**Note:** Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases +

Explorer

numpyarr...

Submit

```python
import numpy as np
r,c = map(int,input().split())

data=[]
if r == 0 and c ==0:
    arr=np.empty((0,0),dtype=int)
else:
    for _ in range(r):
        row=list(map(int,input().split()))
        data.append(row)
        arr=np.array(data)

print(arr)
print(arr.ndim)
print(arr.shape)
print(arr.size)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.1. Numpy: Matrix Operations

00:00

The given code takes two $3 \times 3$ matrices, $\text{matrix_a}$, and $\text{matrix_b}$, as input from the user and converts them into NumPy arrays.

**Task:**

You are required to compute and display the results of the following matrix operations:

1. **Addition** ($\text{matrix_a} + \text{matrix_b}$)
2. **Subtraction** ($\text{matrix_a} - \text{matrix_b}$)
3. **Element-wise Multiplication** ($\text{matrix_a} * \text{matrix_b}$)
4. **Matrix Multiplication** ($\text{matrix_a} \cdot \text{matrix_b}$)
5. **Transpose of Matrix A**

**Input Format:**

- The user will input 3 rows for $\text{matrix_a}$, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for $\text{matrix_b}$, each containing 3 integers separated by spaces.

**Output Format:**

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases +

Explorer

matrixOp... Submit

```python
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])


# Addition
print("Addition (A + B):")
print(matrix_a + matrix_b)
# Subtraction
print("Subtraction (A - B):")
print(matrix_a - matrix_b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
print(matrix_a * matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(matrix_a .dot(matrix_b))
# Transpose
print("Transpose of A:")
print(matrix_a.T)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

05:27

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

**Input Format:**

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

**Output Format:**

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

Explorer

stacking.py

Submit

```python
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
result=np.hstack((arr1,arr2))
print("Horizontal Stack:")
print(result)

# Perform vertical stacking (vstack)
result=np.vstack((arr1,arr2))
print("Vertical Stack:")
print(result)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.3. Numpy: Custom Sequence Generation

04:00

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

**Input Format:**

- The user will input three integer values: start, stop, and step, each on a new line.

**Output Format:**

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS...

Submit

```python
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
sequence=np.arange(start,stop,step)
# Print the generated sequence
print(sequence)
```

Terminal Test cases

Explorer Debugger

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

04:13

You are given two arrays `A` and `B`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

**1. Arithmetic Operations**:
- Compute the element-wise sum, difference, and product of the two arrays.

**2. Statistical Operations**:
- Calculate the mean, median, and standard deviation of array `A`.

**3. Bitwise Operations**:
- Perform bitwise `AND`, bitwise `OR`, and bitwise `XOR` on the arrays (ex: $A_1$ `OR` $B_1$).

**Input Format:**
- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

**Output Format:**
- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases +

Explorer

different...

Submit

```python
import numpy as np

def array_operations(A, B):

    # Convert A and B to NumPy arrays
    arr_A=np.array(A)
    arr_B=np.array(B)

    # Arithmetic Operations
    sum_result = arr_A + arr_B
    diff_result = arr_A - arr_B
    prod_result = arr_A * arr_B

    # Statistical Operations
    mean_A = np.mean(arr_A)
    median_A = np.median(arr_A)
    std_dev_A = np.std(arr_A)

    # Bitwise Operations
    and_result = arr_A & arr_B
    or_result = arr_A | arr_B
    xor_result = arr_A ^ arr_B

    # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))

    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")

    print("Bitwise AND:", ' '.join(map(str, and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.5. Numpy: Copying and Viewing Arrays

05:32

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Sample Test Cases +

Explorer

copyAnd...

Submit

```python
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array =original_array.view()

# Create a copy
copy_array =original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.5. Numpy: Copying and Viewing Arrays

05:32

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Sample Test Cases +

Explorer

copyAnd...

Submit

```python
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array =original_array.view()

# Create a copy
copy_array =original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

## 3.2.5. Numpy: Copying and Viewing Arrays

05:32

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

**Input Format:**

- A single line of space-separated integers.

**Output Format:**

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Sample Test Cases +

Explorer

copyAnd...

Submit

```python
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array =original_array.view()

# Create a copy
copy_array =original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›

CODETANTRA Home Learn Anywhere 202501090019@mitaoe.ac.in Support Logout

### 3.2.7. Student Data Analysis and Operations

11:50

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details**: Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students**: Determine the total number of students in the dataset.
- **Print all student roll numbers**: Extract and print the roll numbers of all students.
- **Print Subject 1 marks**: Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2**: Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3**: Identify the highest marks in Subject 3.
- **Print all subject marks**: Display the marks of all students for each subject.
- **Find total marks of students**: Compute the total marks for each student across all subjects.
- **Find the average marks of each student**: Compute the average marks for each student.
- **Find average marks of each subject**: Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2**: Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3**: Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3**: Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2**: Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2**: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1**: Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2**: Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject**: Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored >=90**: Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order**: Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1**: Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1**: Identify the index positions of students who scored exactly 79 marks in Subject 1.

**Note:** Fill in the missing code to perform the above-mentioned operations.

Explorer

Operatio...

Submit

```python
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)


# All student details
print("All student Details:")
print(" "+str(a))    # FIX 1: correct spacing (no underscore issue)

# Total students
print("Total Students:", a.shape[0])

# Roll numbers
roll = a[:,0]
print("All Student Roll Nos", roll)

# Subject 1
s1 = a[:,1]
print("Subject 1 Marks", s1)

# Subject 2 & 3
s2 = a[:,2]
s3 = a[:,3]

print("Min marks in Subject 2", np.min(s2))
print("Max marks in Subject 3", np.max(s3))

# Subjects
subjects = a[:,1:]
print("All subject marks:", subjects)

# Total
total = np.sum(subjects, axis=1)
print("Total Marks", total)

```

Debugger

Terminal Test cases

‹ Prev Reset Submit Next ›